BLOCKCHAIN-BASED LAND RECORD INTEGRITY, DOCUMENT VERIFICATION AND CONFLICT DETECTION SYSTEM

PROJECT PHASE-I REPORT

Submitted by

JAISHREE S (231901011)

VEDANTH S (231901060)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

RAJALAKSHMI ENGINEERING COLLEGE (AUTONOMOUS), CHENNAI-602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(an Autonomous Institution Affiliated to Anna University Chennai)

BONAFIDE CERTIFICATE

Certified that this Phase – I Report titled "**Blockchain-Based Land Record Integrity, Document Verification and Conflict Detection System**" is the bonafide work of "**JAISHREE S (231901011) and VEDANTH S (231901060)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. P. Tamilselvi
PROFESSOR AND HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai - 602 105

Dr. D. Sivakumar
PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai - 602 105

Internal Examiner **External Examiner**

ABSTRACT

Land and property records are important legal documents. Many of these records are now kept in digital form, but a digital file can be edited after it is stored, and it is often hard to prove that a document shown later is the same as the one registered earlier. This project presents a web-based system for land record integrity, document verification and conflict detection. The current application is built with React and Vite for the frontend, Node.js and Express for the backend, and MongoDB for storage. An administrator can log in, register property records and upload land-related PDF documents. For every uploaded document the system generates a SHA-256 hash, a 64-character value that acts like a fingerprint of the file. During verification, the hash of a submitted document is calculated again and compared with the registered hash, and the result is shown as MATCH, MISMATCH or NOT FOUND. Duplicate Property IDs are rejected at registration. The proposed architecture includes Hyperledger Fabric, a permissioned blockchain, to keep document hashes and transaction details in a tamper-evident ledger, while the current application stores the hashes in MongoDB. Functional testing covered login, property registration, document upload, hash generation, document verification and data persistence after restart. All twelve user interface tests passed.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Dr. Sivakumar D, Professor,** for the encouragement and guidance given throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project, and our parents, friends, all faculty members and supporting staff for their encouragement and support.

JAISHREE S

VEDANTH S

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
	LIST OF ABBREVIATIONS	**ix**
1.	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVES	3
	1.4 SCOPE	4
	1.5 SECURITY REQUIREMENTS AND CONSIDERATIONS	5
	1.6 SDG ALIGNMENT	6
2.	**LITERATURE REVIEW**	**7**
	2.1 EXISTING SYSTEMS AND SECURITY APPROACHES	7
	2.2 RESEARCH GAPS	10
	2.3 PROPOSED SYSTEM	11
	2.3.1 Overview of the Proposed System	11
	2.3.2 Objectives of the Proposed System	11
	2.3.3 Advantages of the Proposed System	12
3.	**SYSTEM DESIGN**	**13**
	3.1 SYSTEM ARCHITECTURE	13
	3.2 MODULE DESIGN	15
	3.3 UML AND DATA FLOW DIAGRAMS	18
	3.4 THREAT MODEL AND ATTACK SURFACE	22

	3.5 SECURITY CONTROLS	26
4.	**SYSTEM DESCRIPTION**	**28**
	4.1 DATA COLLECTION	28
	4.2 DATA PREPROCESSING	29
	4.2.1 Algorithms and Methodology	30
	4.2.2 Result and Discussion	32
5.	**CONCLUSION AND FUTURE ENHANCEMENTS**	**36**
	5.1 CONCLUSION	36
	5.2 FUTURE ENHANCEMENTS	36
	REFERENCES	**37**
	APPENDIX 1	**39**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG alignment	7
3.1	Software requirements	19
3.2	Functional requirements	19
3.3	Security requirements	27
3.4	Threat analysis	28
3.5	Attack surface	29
3.6	Security controls	30

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System architecture	18
3.2	Module interaction diagram	22
3.3	Use case diagram	23
3.4	Activity diagram	24
3.5	Sequence diagram for document verification	25
3.6	Data flow diagram (level 1)	26
3.7	Threat model	28
4.1	Document processing workflow	33
4.2	Hash comparison during verification	36
4.3	Admin login page	40
4.4	Dashboard with statistic cards and recent records	40
4.5	Property registration	40
4.6	Document upload and generated hash	41
4.7	Verification result MATCH & MISMATCH	41

LIST OF ABBREVIATIONS

AES	Advanced Encryption Standard
API	Application Programming Interface
DFD	Data Flow Diagram
HTTP	Hypertext Transfer Protocol
ID	Identifier
IPFS	InterPlanetary File System
JSON	JavaScript Object Notation
MSP	Membership Service Provider
NER	Named Entity Recognition
OCR	Optical Character Recognition
PDF	Portable Document Format
REST	Representational State Transfer
SDG	Sustainable Development Goal
SHA	Secure Hash Algorithm
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
UI	User Interface
UML	Unified Modeling Language

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Land is one of the most valuable assets that a person or family can own. Ownership of land is established through records and documents such as sale deeds, title documents, survey records and tax receipts. These records provide important information about the ownership and legal status of a property and are required during activities such as buying, selling, inheriting, mortgaging and taxation.

With the increasing use of digital technologies, land records are being maintained and accessed in electronic form. Digital records make information easier to store, search, retrieve and share between authorised users. They can also reduce the effort involved in maintaining and accessing large volumes of physical documents.

However, maintaining land records digitally also creates challenges related to **data integrity, document authenticity and consistency**. Digital documents can be modified, replaced or duplicated without physical evidence of the change. When records are maintained by different offices or systems, comparing information between them can also become difficult. An incorrect or outdated record may therefore affect the verification of property ownership and lead to disputes during land-related transactions.

Another important concern is the verification of documents presented as proof of ownership. A document submitted during a transaction must correspond to the original registered record. Manual checking of documents can be time-consuming, particularly when a large number of records have to be examined.

Therefore, maintaining the **integrity, authenticity and consistency of land records** is an important requirement in digital land administration. A reliable system should support secure record management, make changes traceable and provide an effective method for verifying documents and identifying conflicting records.

This creates the need for a technology-based approach that can provide greater confidence in the integrity of digital land records while supporting efficient document verification and record management.

1.2 PROBLEM STATEMENT

Digital land records are easy to store and share, but there is no simple and dependable way to confirm, at the time of use, that a land document is the same as the one that was registered. In addition, records that conflict with each other are usually found late.

A registered document can be modified after it is stored, and an ordinary database may not reveal that the content has changed. A person who receives a document cannot easily compare it with the registered copy, as manual comparison takes time and can miss small edits. A property can also be entered more than once, or related records may disagree on details such as the owner name, area or survey number. Furthermore, if the registered hash is kept in a database that can be edited, the reference value used for checking can itself be changed.

Therefore, the project aims to build a system that records a SHA-256 hash for every registered document, allows an authorised user to verify a submitted document against the registered hash and receive a clear result, flags possible conflicts for officer review, and keeps the registered hashes in a record that is difficult to alter without detection. The system only reports whether a document matches its registered version and whether records appear to conflict; it does not decide who legally owns a property.

1.3 OBJECTIVES

The project aims to develop a web application for registering property records with unique Property IDs and uploading land-related PDF documents. The system validates the uploaded files, generates a SHA-256 hash for each document, and stores the hash along with the corresponding document and property details. It also restricts access to the application to an authenticated administrator.

The system aims to provide reliable document verification by recalculating the hash of a submitted document and comparing it with the registered hash. Based on the comparison, the system provides a clear result as **MATCH, MISMATCH or NOT FOUND**. It also identifies possible conflicts between property records using predefined validation rules, including the detection and rejection of duplicate Property IDs.

The project also aims to design a **proposed blockchain-based architecture** in which document hashes and important transaction details can be recorded using Hyperledger Fabric. The completed application is tested using functional test cases, and the actual test results are documented to evaluate the implemented functionalities.

1.4 SCOPE

The scope of the project is divided into the current application, the proposed design, and the limitations of the system.

Current application: The current application covers administrator login, a dashboard with five statistic cards and three tables of recent records, registration and listing of property records with rejection of duplicate Property IDs, upload of PDF documents with rejection of non-PDF files, SHA-256 hash generation and storage of the hash in MongoDB, a document list, and document verification with **MATCH, MISMATCH and NOT FOUND** results.

Proposed design: The proposed design extends the system by including the recording of document hashes and important transaction details on a Hyperledger Fabric ledger, maintaining a history of ownership transactions for each property, applying conflict rules for owner mismatch, area mismatch, duplicate survey numbers, invalid ownership transfers and duplicate document hashes, and sending detected conflicts to an authorised officer for review.

Limitations: The system is designed to work alongside the existing registration process and does not replace it. A **MATCH** result only indicates that the submitted file is identical to the registered file; it does not prove that the contents of the document are true or legally valid. A detected conflict is only a flag for review, and the system does not label any record as fraudulent or determine legal ownership. Only PDF documents are accepted. The application has been tested for functional correctness in a development environment, while performance, load and penetration testing have not been carried out. The test results presented in this report are from the MongoDB-based application and do not include Hyperledger Fabric transactions.

1.5 SECURITY REQUIREMENTS AND CONSIDERATIONS

A security requirement states what the system must protect, while a security control is the mechanism used to provide that protection. This section describes the security requirements relevant to the project and indicates how they are addressed in the current application. The corresponding controls are discussed in Section 3.5.

Authentication: Only an authorised person should be able to access the application. The current application provides an administrator login. An invalid password is rejected, while a valid login opens the dashboard.

Authorization: A user should be able to perform only the actions permitted for the assigned role. In the current application, the implemented functions are accessed through the administrator account. Separate roles with limited permissions, such as a verifier who can verify documents and an officer who reviews conflicts, are part of the proposed design.

Integrity: A registered document should not be modified without the change being detectable. The application generates a SHA-256 hash for each uploaded document and stores the hash with the corresponding document record.

Tamper detection: If a registered document is altered, the system should be able to detect the change. During verification, the hash of the submitted document is calculated again and compared with the registered hash. If the document has been modified, the hashes differ and the verification result is displayed as **MISMATCH**, with the computed and registered hashes shown for comparison.

Secure document handling: Uploaded files should be validated before they are accepted by the system. The current application accepts PDF documents and rejects files that are not in PDF format.

Data protection: Land documents may contain personal and property information. In the proposed architecture, the documents remain in off-chain storage, while only the document hash and selected transaction details are intended to be recorded on the Hyperledger Fabric ledger.

Auditability: It should be possible to determine when records were created or documents were uploaded. The current application records the upload time of documents and displays recent records on the dashboard. A complete and tamper-evident transaction history is part of the proposed Hyperledger Fabric architecture.

Reliability of stored data: Registered property and document records should remain available after system or database restarts. The MongoDB data was checked after restarting the database and backend, and the previously stored records remained available.

A limitation of the current application is that the registered document hash is stored in MongoDB. A person with direct access to the database could potentially modify the stored hash. In such a case, modification of the document can be detected through hash comparison, but modification of the reference hash itself

may not be detected. Recording the hash on a blockchain ledger is therefore proposed as an additional tamper-evident mechanism.

1.6 SDG ALIGNMENT

The Sustainable Development Goals (SDGs) of the United Nations that relate to this project are shown in Table 1.1. The contribution is at the level of a student prototype, and the table states it in those terms.

Table 1.1 SDG alignment

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 16: Peace, Justice and Strong Institutions	Target 16.6: Develop effective, accountable and transparent institutions at all levels	The system gives a way to check a land document against its registered hash. This supports transparent and accountable record keeping.
2	SDG 16: Peace, Justice and Strong Institutions	Target 16.5: Substantially reduce corruption and bribery in all their forms	A changed document is detected at verification, and possible conflicts are flagged for officer review. This makes it harder to pass an altered document unnoticed.
3	SDG 1: No Poverty	Target 1.4: Equal rights to economic resources, including ownership and control over land and other forms of property	Reliable verification of land documents helps owners protect their property records. The contribution is indirect.
4	SDG 9: Industry, Innovation and Infrastructure	Target 9.b: Support domestic technology development, research and innovation	The project applies hashing and a permissioned blockchain design to a public record problem using web technologies.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews fifteen research papers related to the project. They cover blockchain-based land registration, verification of digital documents using hashes, detection of conflicts or unusual records, and access control on permissioned blockchains. Six of the papers were studied from their full text (Imam et al 2021; Irimia and Iftene 2024; Khanna et al 2026; Sen et al 2021; Stefanović et al 2022; Zein and Twinomurinzi 2024). The others were studied from their published abstracts and bibliographic details, and this report makes no claim about them beyond that information.

2.1 EXISTING SYSTEMS AND SECURITY APPROACHES

Land registration systems have increasingly explored blockchain technology to improve the integrity, traceability and sharing of property records. Sen et al. (2021) proposed a Hyperledger Fabric framework in which the Registration Authority, Land Authority and Municipal Authority act as peers. The title deed is stored off-chain using the InterPlanetary File System (IPFS), while the ledger maintains the reference and transaction details. Ownership transfer requires approval from all three authorities. The framework was tested using Hyperledger Caliper in both single-host and multi-host environments. Its security approach combines permissioned access, approval from multiple authorities and off-chain document storage. However, the work does not describe a process in which a submitted document is hashed again and compared with a registered hash, and it does not include rule-based conflict detection.

Khanna et al. (2026) addressed the Indian context in which records belonging to a land parcel may be distributed across several offices. Their prototype uses Hyperledger Fabric with separate membership service providers for different administrative roles. Mutations, encumbrances and disputes are maintained as states in the chaincode, while documents are stored off-chain in encrypted form. The authors evaluated throughput and latency using Hyperledger Caliper. They

also identify limitations in their work, including the use of published aggregate figures for the workload, a placeholder court role and simulated connections to state land portals. The system is presented as a validation and audit layer rather than a replacement for the statutory land record. However, disputes are represented as a general state raised by a party rather than being detected through predefined conflict rules.

Zein and Twinomurinzi (2024) designed a Hyperledger Fabric architecture for land registration in Sudan using an action design research approach. The architecture involves the Urban Planning Authority, Land Registration Authority and a bank, with information shared through separate channels. Algorithms are provided for land assignment and land sale, demonstrating how blockchain can address problems such as double selling and record tampering. However, the authors state that the prototype had not yet been evaluated, and document hashing and document verification were not specified. Dcunha et al. (2023) also proposed Hyperledger Fabric for simplifying land registration in India, supporting the use of a permissioned blockchain approach for land-related applications.

Stefanović et al. (2022) focused on transactions within a land administration system and developed a Solidity smart contract based on features of the ERC-20 and ERC-721 token standards. The system supports shared ownership, partial transfers, restricted transfers, splitting and merging of real estate. It was deployed on the Ethereum Ropsten test network and tested using 1,000 simulated transactions. Although a document hash is associated with a transfer, the system does not provide a mechanism for verifying a submitted document against the stored hash. The work also uses a public blockchain and does not address rule-based conflict detection.

Other studies have combined blockchain with additional techniques. Naguji et al. (2024) presented GreenLand, an agricultural land registration scheme that combines blockchain with artificial intelligence models to address forged or fraudulent records. Nwankwo et al. (2026) proposed BERBLOM using

Hyperledger Besu, ERC-4907 tokens and unique parcel identifiers to address inconsistent documents and double allocation of land. These studies demonstrate that duplicate and inconsistent land records are recognised challenges. However, their approaches rely on artificial intelligence or tokenisation, whereas the present project uses predefined rules for conflict identification.

Document integrity and verification have also been studied independently of land registration. Imam et al. (2021) developed DOC-BLOCK, a web application that uses SHA-256 hashing to verify digital documents. The hash of an uploaded document is stored through a smart contract on Ethereum, while the document is stored on IPFS. During verification, the document is hashed again and compared with the stored value. The study demonstrates that even a small modification to a document produces a different hash. This verification mechanism is relevant to the present project, although DOC-BLOCK focuses on general documents and does not include property records, ownership information or conflict detection.

Irimia and Iftene (2024) proposed a microservice-based system for notarising, signing and sharing official documents. Their approach combines document hashing, AES encryption, human notary confirmation, IPFS storage and a private Ethereum-based network. The study discusses issues such as dependence on IPFS nodes, operating costs and the conflict between permanent blockchain records and the requirement to remove personal information. The work supports the use of storing only selected information on a ledger and retaining human involvement in sensitive decisions.

Mishra and Ganesan (2025) studied shadow attacks on PDF documents in e-governance, where the visible content of a signed PDF can be modified. They proposed a blockchain-based validation framework that records document hashes to detect such changes. Kumutha and Jayalakshmi (2021) developed a Hyperledger Fabric framework for academic certificates using SHA-256 hashing and digital watermarking. Liu and Chang (2024) combined SHA-256, Hyperledger Fabric and IPFS for retrieving and verifying remote sensing images. Although these works address different application domains, they demonstrate

the use of hashing and permissioned blockchain technologies for protecting and verifying stored digital records.

Conflict detection and access control are additional aspects considered in related systems. Nagaraja and Shushma (2025) proposed a land-record framework that applies optical character recognition, named entity recognition and anomaly detection before records are stored using Ethereum and IPFS. The approach checks records before they become permanent, but it depends on artificial intelligence models, whereas the present project uses predefined rules whose results can be directly explained. Alotaibi et al. (2025) proposed a conceptual Hyperledger Fabric model for transparent government records with different access levels for citizens, auditors and regulators. Pari et al. (2022) implemented role-based access control using Hyperledger Fabric for healthcare records. These works demonstrate the relevance of role-based access in permissioned systems and support the proposed administrator, verifier and officer roles in this project.

2.2 RESEARCH GAPS

The following gaps were identified from the fifteen papers reviewed. These gaps relate to how different features are combined, and no claim is made that any individual feature is new. First, document checking is generally treated separately from land records. Land registration systems based on permissioned blockchains store documents or their references, but they do not describe a function in which a user submits a document and the system recalculates its hash for comparison.

Second, conflict handling is either represented as a general dispute state, identified using machine learning techniques, or addressed through token-based identifiers. As far as the reviewed literature shows, a small set of predefined rules such as owner mismatch, area mismatch, duplicate survey number, invalid ownership transfer and duplicate document hash, with the results sent to an authorised officer for review, is not described. Role separation is also mainly demonstrated in general government systems and other domains such as

healthcare, while the reviewed land-registration studies do not describe its combination with direct document verification.

Finally, the evaluation of combined systems is limited, with some proposed systems not evaluated and others tested using simulated workloads or placeholder components. The present project addresses these gaps by combining document verification, property record management and rule-based conflict identification in a single application, while also providing a proposed blockchain architecture for the remaining features. The document verification function and duplicate Property ID validation are implemented and tested in the current application, while ownership history, additional conflict rules, role separation and Hyperledger Fabric-based hash recording are included as proposed extensions.

2.3 PROPOSED SYSTEM

2.3.1 Overview of the Proposed System

The proposed system is a web-based verification layer for land records. It works beside the existing registration process. An administrator registers a property and uploads the related documents. The system generates a SHA-256 hash for each document, keeps the document in off-chain storage and records the hash with the property and document details. Later, any submitted copy of the document can be verified by generating its hash again and comparing the two values. Records are also checked against predefined rules, and possible conflicts are sent to an authorised officer for review.

The proposed architecture uses Hyperledger Fabric as the blockchain component for recording document hashes and important transaction details, while the current application stores the document hashes in MongoDB.

2.3.2 Objectives of the Proposed System

1. To connect hash-based document verification directly with property records.

2. To give a clear verification result that separates an unchanged document, a changed document and an unregistered document.
3. To detect possible conflicts with fixed rules that can be explained, instead of AI models.
4. To leave the final decision on a conflict to an authorised officer.
5. To keep documents off-chain and place only the hash and essential details on a permissioned blockchain.
6. To work beside the existing registration system without replacing it.

2.3.3 Advantages of the Proposed System

The proposed system has the following advantages. A document is verified by submitting it, so the check is quick and repeatable and no manual comparison is needed. Any change in the content of the file gives a different SHA-256 hash, so even small changes are detected. The results MATCH, MISMATCH and NOT FOUND tell the user exactly what was found, and for a mismatch both hashes are shown. A duplicate Property ID is rejected at the time of entry, so duplicates are stopped early. In the proposed architecture only the hash and selected details go on the ledger and the documents stay off-chain, which respects privacy. A conflict is only flagged and an officer reviews it, so a genuine record is not wrongly labelled as fraud. Hash comparison and fixed rules are also simple to understand and to audit.

CHAPTER 3

SYSTEM DESIGN

This chapter describes the design of the system. The current application and the proposed blockchain component are marked separately in every diagram. Solid lines show the current application, and dashed lines show proposed parts.

3.1 SYSTEM ARCHITECTURE

The system follows a three-layer web architecture, as shown in Figure 3.1.

Presentation layer: The user interface is built with React, a JavaScript library for building web pages from reusable components, and Vite, a tool that runs and builds the frontend project. It runs in the web browser and contains the login page, the dashboard and the pages for property registration, document upload and document verification.

Application layer: The backend is built with Node.js and the Express framework. It provides a REST API, which is a set of web addresses that the frontend calls to send and receive data. The backend handles admin authentication, property records, PDF validation, SHA-256 hash generation, verification and the dashboard figures. In the development environment, the API runs on port 5001 because port 5000 was already in use.

Data layer: MongoDB, a document-oriented database, stores the property records, the document details and the SHA-256 hashes. It maintains the information required for property registration and document verification. The stored records can be retrieved by the backend when displaying property details, document information and verification results.

Proposed blockchain component: The proposed architecture includes a Hyperledger Fabric ledger on which the document hash, property ID, document ID and timestamp are to be recorded. The documents themselves

remain off-chain. The current application stores the document hashes in MongoDB.

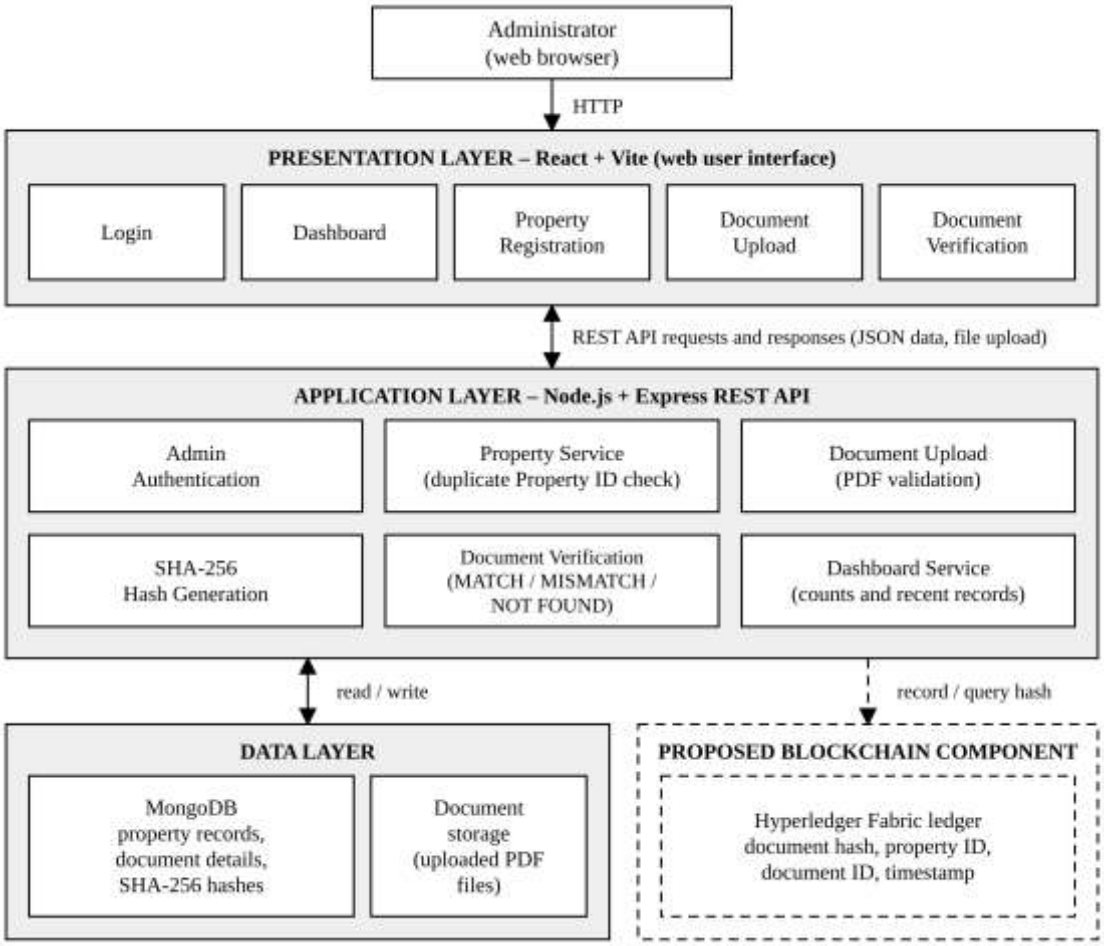


Figure 3.1 System architecture

A typical request passes through the layers as follows. The administrator performs an action in the browser. The React interface sends a request to the Express API. The backend validates the input, carries out the operation, reads or writes data in MongoDB and returns a response, which the interface then displays.

Table 3.1 Software Requirements

Category	Software
Operating system	**Windows / macOS / Linux**
Frontend	**React with Vite**
Backend	**Node.js with Express**
Database	**MongoDB**
Hashing	**SHA-256**
Browser used for UI testing	**Google Chrome**
API testing	**CURL**
Proposed blockchain platform	**Hyperledger Fabric (proposed component)**

3.2 MODULE DESIGN

The functional requirements of the system are listed in Table 3.3.

Table 3.2 Functional Requirements

ID	Requirement	Status
FR1	The administrator shall log in with valid login details. An invalid password shall be rejected.	Implemented
FR2	The dashboard shall show summary statistics and recent records.	Implemented
FR3	The administrator shall register a property with a unique Property ID.	Implemented
FR4	The system shall reject a duplicate Property ID.	Implemented
FR5	The administrator shall upload a PDF document for a property. Non-PDF files shall be rejected.	Implemented
FR6	The system shall generate a SHA-256 hash for each uploaded document and store it.	Implemented
FR7	The system shall list uploaded documents with property, file name, file size, shortened hash and upload time.	Implemented
FR8	The system shall verify a submitted document and report MATCH, MISMATCH or NOT FOUND.	Implemented

ID	Requirement	Status
FR9	The system shall record the document hash and transaction details on Hyperledger Fabric.	Proposed
FR10	The system shall keep the ownership transaction history of a property.	Proposed
FR11	The system shall check records against the predefined conflict rules.	Partly implemented (duplicate Property ID)
FR12	Detected conflicts shall be sent to an authorised officer for review.	Proposed

The system is divided into functional modules that work together to support property registration, document management, document verification and conflict identification. Figure 3.2 shows the interaction between these modules.

1. Admin Authentication Module: This module controls entry to the application. The administrator enters the login details on the login page and the application checks them. If the password is wrong, the login is rejected. If the details are correct, the dashboard opens.

2. Dashboard Module: After login, the dashboard gives an overview of the system. It contains five statistic cards and three tables of recent records.

3. Property Registration Module: The administrator enters the details of a property, including a unique Property ID. When the record is saved successfully, the form is cleared and the new property appears in the property list. If the Property ID already exists, the system rejects the duplicate entry.

4. Document Upload and Storage Module: The administrator selects a registered property and uploads a land-related document in PDF format. A valid PDF is accepted, while non-PDF files are rejected. The document list shows the property, file name, file size, a shortened form of the hash and the upload time.

5. Hash Generation Module: For each accepted document, this module calculates the SHA-256 hash of the file. The hash is displayed in full as 64 hexadecimal characters and is stored with the document details in MongoDB.

6. Document Verification Module: The user submits a document to be checked. The module calculates its SHA-256 hash and compares it with the registered hash. The result is **MATCH** if the hashes are equal, **MISMATCH** if a registered record exists but the hashes differ, and **NOT FOUND** if there is no registered record for the document. For a MISMATCH, the computed hash and the registered hash are shown side by side.

7. Conflict Detection Module: This module checks records against predefined rules. In the current application, duplicate Property ID detection is implemented. The rules for owner mismatch, area mismatch, duplicate survey number, invalid ownership transfer and duplicate document hash are proposed. In the proposed design, a detected conflict is sent to an authorised officer for review, and the system does not label it as fraud.

8. Blockchain Integrity Record Module (proposed): In the proposed architecture, this module records the document hash, Property ID, document ID and timestamp on a Hyperledger Fabric ledger through a smart contract, which Fabric calls chaincode. Verification can then use the hash kept on the ledger as the reference value. The current application keeps this information in MongoDB.

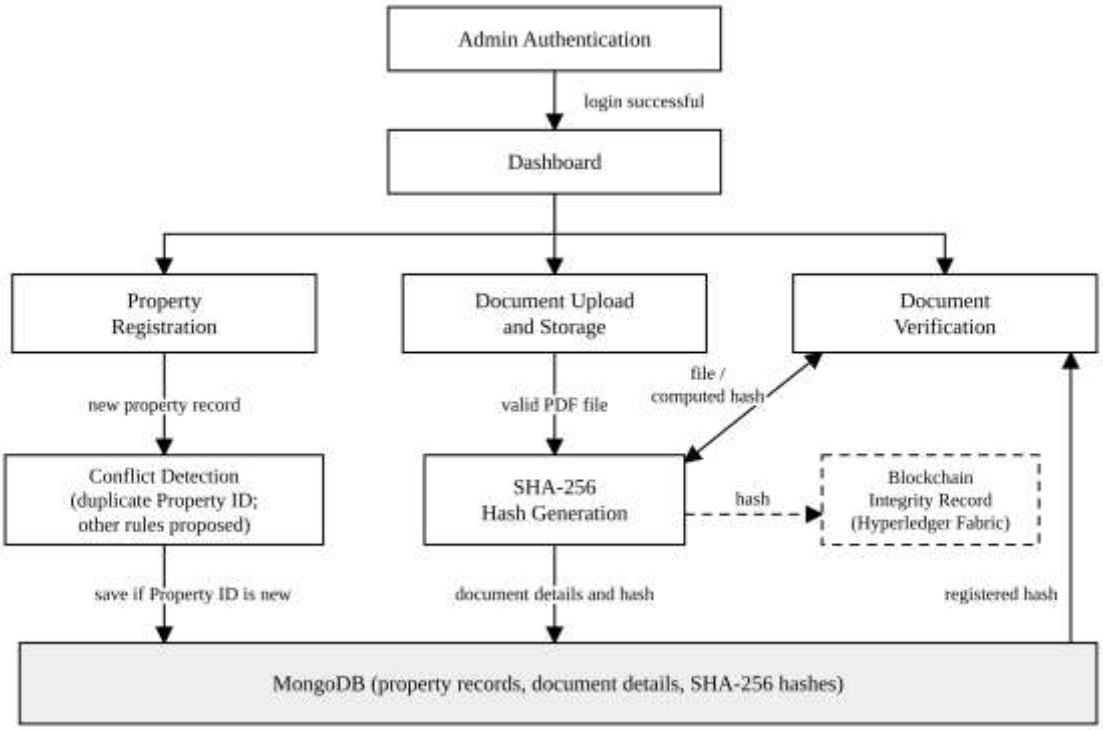


Figure 3.2 Module interaction diagram

3.3 UML AND DATA FLOW DIAGRAMS

3.3.1 Use Case Diagram

A use case diagram shows who uses the system and what they can do. Figure 3.3 shows the administrator as the actor of the current application. The administrator can log in, view the dashboard, register a property, upload a PDF document, view the lists and verify a document. Uploading and verifying both include hash generation, and registering a property includes the duplicate Property ID check. The verifier, the authorised officer, the review of flagged conflicts and the recording of the hash on the Fabric ledger are drawn with dashed lines because they are proposed.

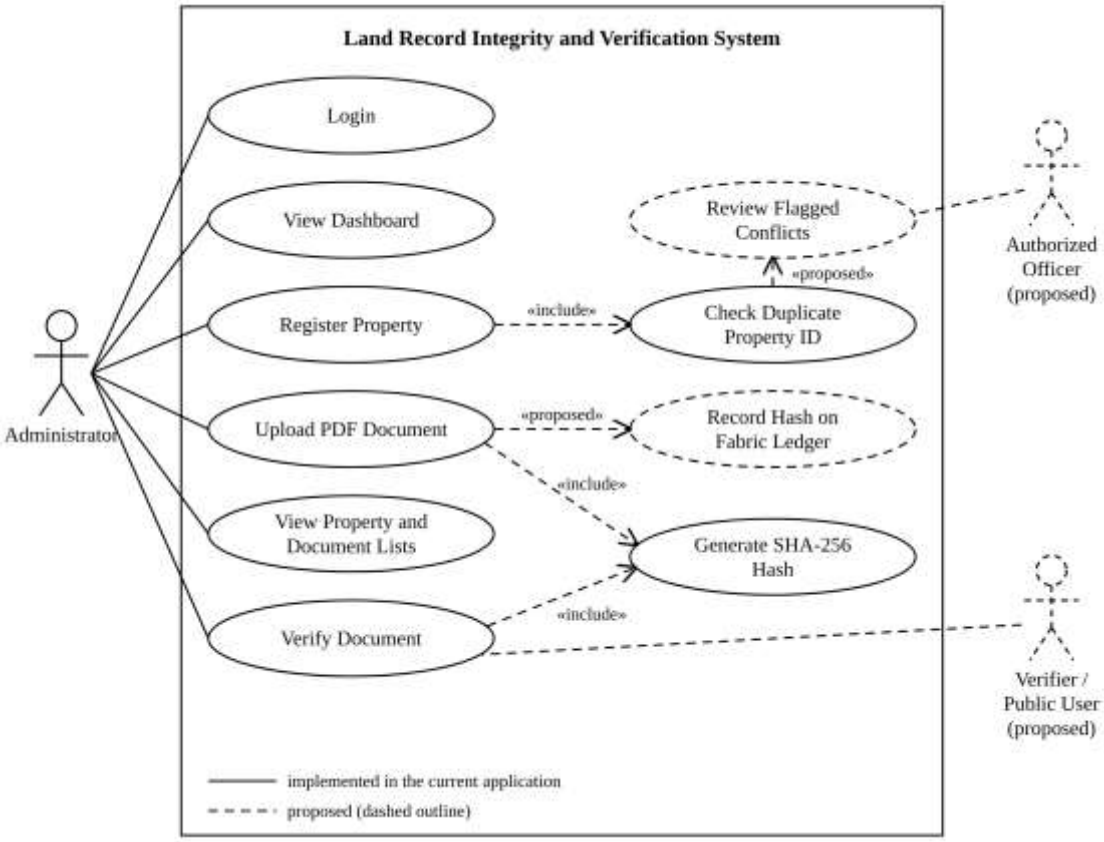


Figure 3.3 Use case diagram

3.3.2 Activity Diagram

An activity diagram shows the order of steps and decisions. Figure 3.4 starts with the login. After a valid login the administrator chooses one of three actions. When a property is registered, the Property ID is checked and a duplicate is rejected. When a document is uploaded, a file that is not a PDF is rejected. otherwise the hash is generated, stored and displayed. When a document is verified, its hash is computed and compared, and one of the three results is shown.

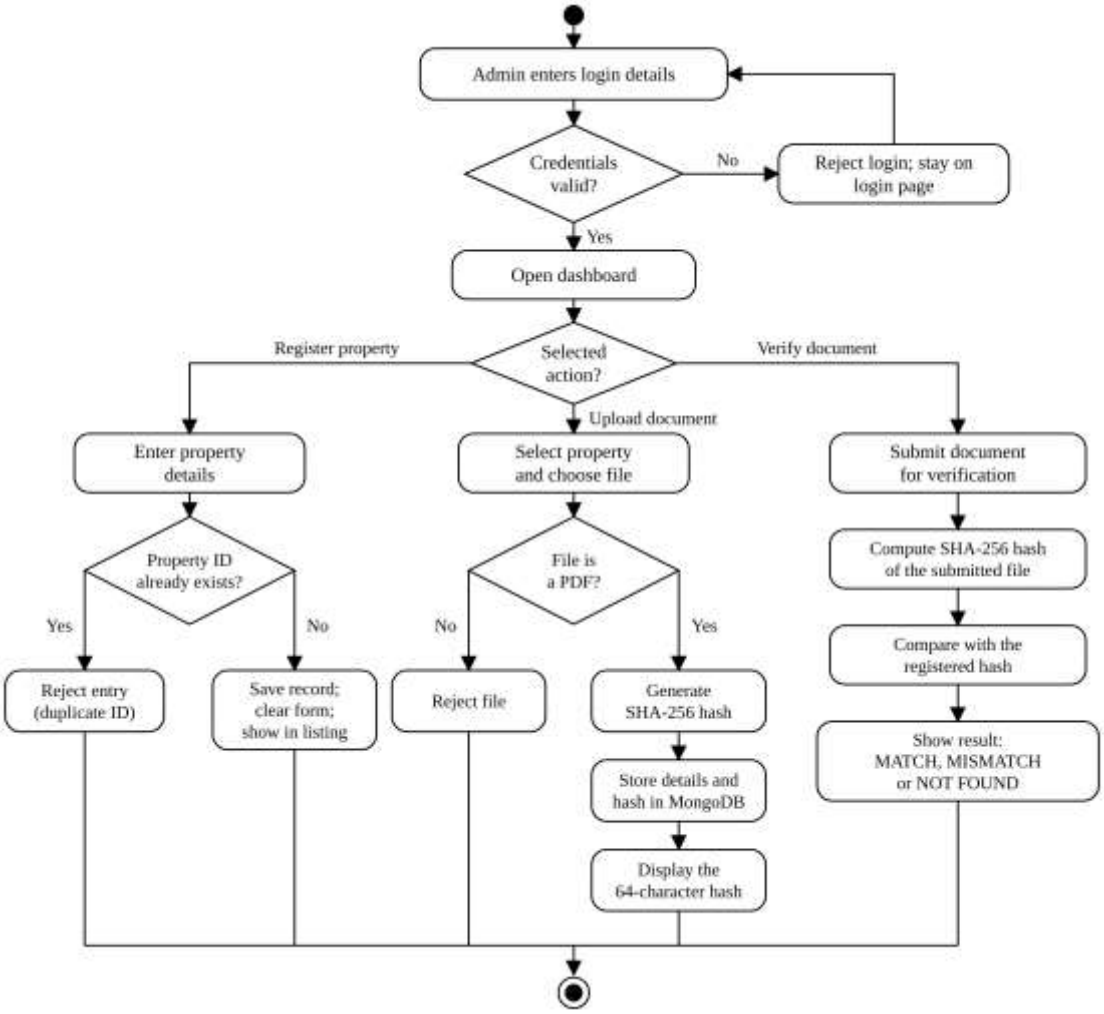


Figure 3.4 Activity diagram

3.3.3 Sequence Diagram

A sequence diagram shows the messages exchanged between parts of the system in time order. Figure 3.5 shows document verification. The administrator submits a document through the React interface, which sends a verification request to the Express API. The API obtains the SHA-256 hash of the submitted file, looks up the registered record in MongoDB, compares the two hashes and returns the result to the interface.

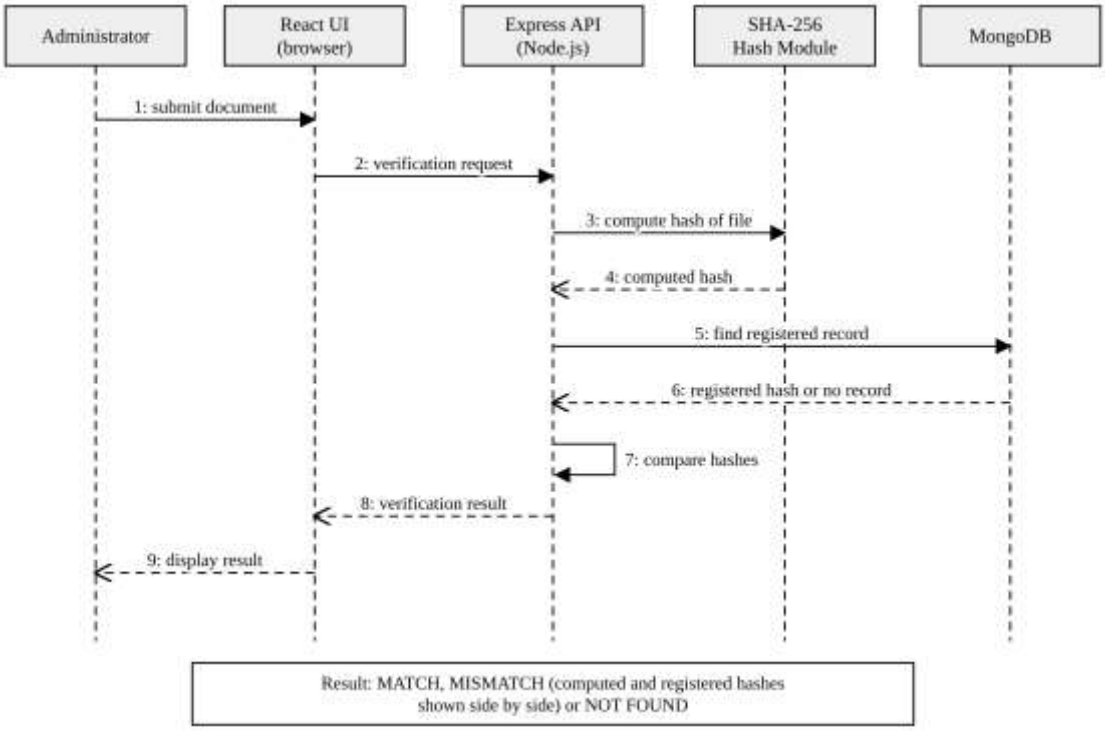


Figure 3.5 Sequence diagram for document verification

3.3.4 Data Flow Diagram

A data flow diagram (DFD) shows how data moves between the user, the processes and the data stores. Figure 3.6 is the level 1 DFD. It has five processes: authenticate admin, register property, upload document and generate hash, verify document, and show dashboard. Property records and document records with hashes are the main data stores. The Fabric ledger is shown as a proposed data store.

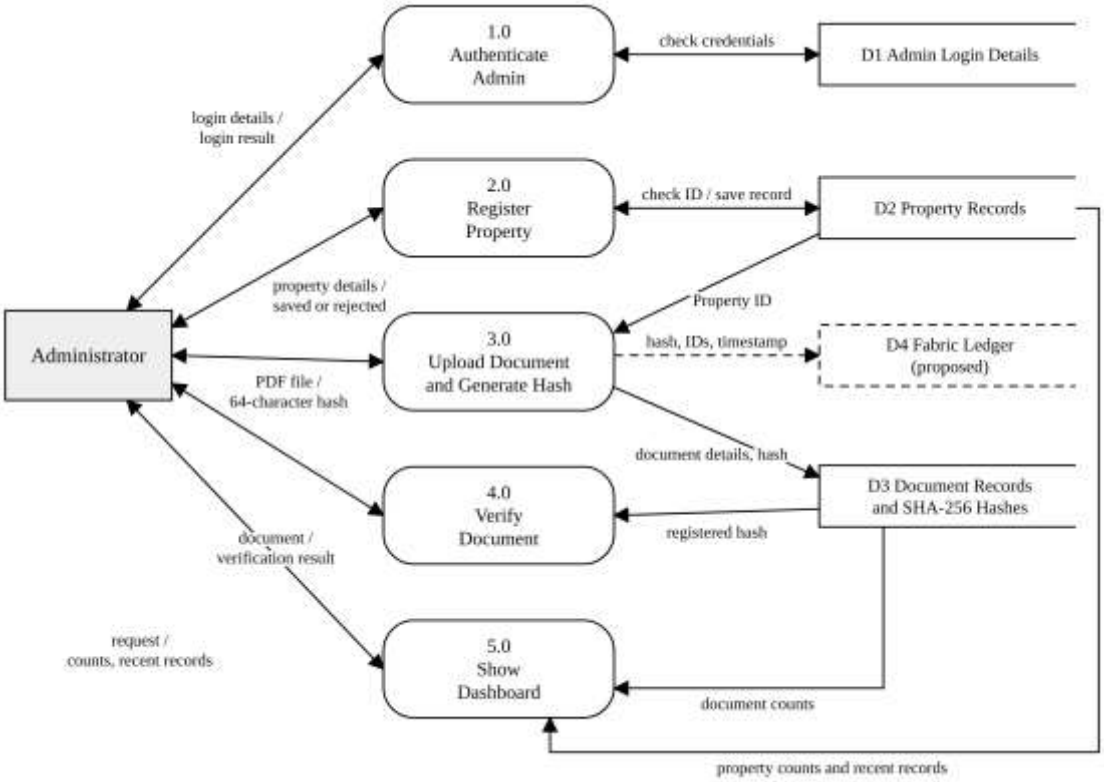


Figure 3.6 Data flow diagram (level 1)

3.4 THREAT MODEL AND ATTACK SURFACE

A threat is something that could go wrong and harm the system or its data. The attack surface is the set of places where an attacker can interact with the system. A threat model brings these together by identifying what must be protected, who might attack the system and how the system could be affected.

Assets and security requirements: The **Assets and security requirements.** The things that need protection are the registered documents, the SHA-256 hashes, the property records, the administrator login details and the verification results. Table 3.4 states the security requirement for each.

Table 3.3 Security Requirements

ID	Requirement	Asset protected	What the system needs to ensure
SR1	Authentication	Administrator account	Only the administrator can log in
SR2	Authorization	Application functions and API	Each role can perform only its permitted actions
SR3	Integrity	Registered documents and hashes	A registered document cannot change unnoticed

ID	Requirement	Asset protected	What the system needs to ensure
SR4	Tamper detection	Submitted documents	A changed document is detected at verification
SR5	Secure document handling	Upload function and server storage	Only the expected file type is accepted and stored safely
SR6	Data protection	Personal details in documents and records	Documents are not exposed to unauthorised persons
SR7	Auditability	Record history	It is possible to see when records were created or changed
SR8	Reliability	Stored records and hashes	Records remain available after a restart

Threat actors: Four types of actors are considered. An outside attacker may try to reach the application or its API. A person may attempt to present an altered document as genuine. An insider may have direct access to the database or server and may attempt to modify stored records. An authorised user may also make an unintentional mistake, such as entering the same property more than once.

Trust boundaries: Information received from the user side, including form data and uploaded files, is treated as untrusted until it is checked by the server. The backend validates the received information before processing it or storing it in the database.

The threats were identified with the help of the STRIDE categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege. Table 3.5 lists the threats with their present status. Threats T1 to T7 are marked in Figure 3.7, and T8 and T9 apply to the system as a whole.

Table 3.4 Threat Analysis

ID	Threat	STRIDE category	Current status
T1	**Password guessing or use of stolen admin login details**	**Spoofing**	**Invalid password is rejected (tested). Repeated-login attempt limiting is not implemented/tested.**
T2	**Upload of a non-PDF or harmful file**	**Tampering, Denial of service**	**Non-PDF files are rejected (tested). File-size limiting and additional content checks are not implemented/tested.**
T3	**Altered document presented as genuine**	**Tampering**	**Detected. Verification returns MISMATCH (tested).**
T4	**API requests sent without a valid login**	**Elevation of privilege**	**Unauthorized API access is checked at the server side. Invalid access is rejected.**
T5	**Stored document or hash changed by a person with database access**	**Tampering**	**Not detected by the current application if the stored hash is changed. Addressed in the proposed design by recording the hash on Hyperledger Fabric.**
T6	**Duplicate or conflicting property entry**	**Tampering**	**Duplicate Property ID is rejected (tested). Other conflict rules are proposed.**
T7	**Loss of stored data after a restart or failure**	**Denial of service**	**Persistence after restart is verified (tested). Backup and recovery mechanisms are not implemented/tested.**
T8	**A user denies having registered or changed a record**	**Repudiation**	**Upload time is recorded. A full transaction history is proposed through the ledger.**
T9	**Exposure of documents or personal details**	**Information disclosure**	**Additional file-level access controls and HTTPS are not implemented/tested in the current development setup.**

The **Attack surface.** Table 3.6 lists the points at which the system accepts input or can be reached.

Table 3.5 Attack Surface

Entry point	Input accepted	Possible misuse
Login page and login API	Login details	Password guessing; use of stolen login details

Entry point	Input accepted	Possible misuse
Property registration form and API	Property details	Invalid, duplicate or conflicting entries; harmful input text
Document upload function	Files	Non-PDF or oversized files; harmful file content
Document verification function	Files	Altered documents; repeated large uploads
REST API (port 5001 in development)	HTTP requests	Direct requests that bypass the user interface
MongoDB database and document storage	Database connections, file access	Direct change or deletion of records, documents or hashes
Hyperledger Fabric network (proposed)	Chaincode transactions	Access by a participant that has not been approved

Attack surface: The main attack surfaces include the login page and login API, property registration form and API, document upload function, document verification function, REST API and MongoDB database and document storage. The login interface accepts authentication details and may be targeted through password guessing or stolen credentials. The property registration interface accepts property information and may receive invalid, duplicate or conflicting entries.

The document upload and verification functions accept files and may be exposed to altered, oversized or unexpected files. The REST API, which runs on port 5001 in the development environment, accepts HTTP requests and therefore requires appropriate server-side validation and authorization. MongoDB and document storage contain records, documents and hashes and must therefore be protected from unauthorised direct access or modification. The proposed Hyperledger Fabric network also represents an attack surface because its chaincode transactions must be accessible only to approved participants.

3.5 SECURITY CONTROLS

A security control is the way in which the system actually meets a requirement. Table 3.7 maps the requirements to their controls and shows whether each control is implemented and tested, still to be confirmed from the project code, or proposed.

Table 3.6 Security Controls

Requirement	Control	Status
Authentication	**Administrator login; invalid password rejected**	**Implemented and tested**
Authorization	**Administrator access to the current application**	**Implemented**
Authorization	**Separate verifier and officer roles**	**Proposed**
Integrity	**SHA-256 hash generated and stored for every uploaded document**	**Implemented and tested**
Tamper detection	**Hash calculated again and compared during verification; MISMATCH shows the computed and registered hashes**	**Implemented and tested**
Tamper detection	**Document hash recorded on a tamper-evident Hyperledger Fabric ledger**	**Proposed**
Secure document handling	**Only PDF files accepted; non-PDF files rejected**	**Implemented and tested**
Secure document handling	**File size limit and additional content checks**	**Not implemented/tested**
Data protection	**Documents kept off-chain; hash and selected details recorded on the proposed ledger**	**Proposed**
Data protection	**HTTPS and additional protection of stored files**	**Not implemented/tested**
Record consistency	**Duplicate Property ID rejected**	**Implemented and tested**
Record consistency	**Further conflict rules and officer review**	**Proposed**
Auditability	**Upload time stored and displayed for each document**	**Implemented and tested**
Auditability	**Transaction history on the proposed ledger**	**Proposed**

Implemented controls: The controls implemented and tested protect against the most direct threats to the current application. A changed document is detected through hash comparison, an incorrect password is rejected, a non-PDF file is not accepted, a duplicate Property ID is rejected, and stored data remains available after a restart.

Remaining risk: The main remaining risk is the possibility of direct modification of the stored reference hash. Because the current application stores the registered hash in MongoDB, a person with direct database access could modify the stored hash. The proposed Hyperledger Fabric ledger is intended to address this limitation by maintaining a tamper-evident record of the document hash.

CHAPTER 4

SYSTEM DESCRIPTION

This chapter describes the work completed in the current application: the data it handles, the way a document is processed, the methods used and the results of testing. The application consists of a React and Vite frontend, a Node.js and Express backend and a MongoDB database.

4.1 DATA COLLECTION

This is not a machine learning project, so there is no training dataset. The data used by the system is entered by the administrator through the web interface or generated by the system during document processing. The main data handled by the system includes the Property ID, property and ownership details, land-related PDF documents, document metadata, SHA-256 hashes, verification information and administrator login details.

The Property ID acts as a unique identifier for a property, and duplicate Property IDs are rejected. Property details are entered by the administrator, while land-related documents are uploaded and linked to the registered property. During document upload, the system captures information such as the file name, file size, upload time and associated property.

For each accepted document, the system generates a SHA-256 hash and stores it with the document details. During verification, the system generates a computed hash and compares it with the registered hash to produce one of three results: **MATCH, MISMATCH or NOT FOUND**. The stored records remained available after the database and backend were restarted.

4.2 DATA PREPROCESSING

The document preprocessing workflow consists of file selection, PDF validation, hash generation, storage and preparation for verification. No machine learning preprocessing, text extraction or image processing is involved.

4.1 shows the steps applied to a document from the moment it is uploaded until it is ready for verification. No machine learning preprocessing, text extraction or image processing is involved.

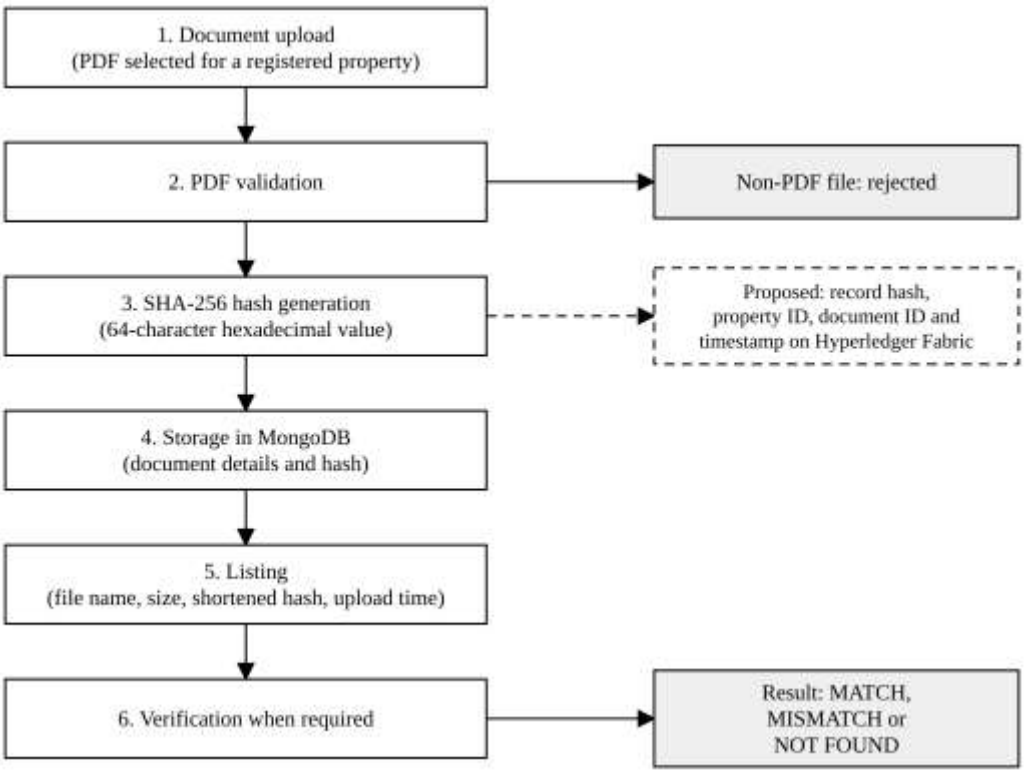


Figure 4.1 Document processing workflow

1. **Document upload.** The administrator selects a registered property and chooses a file.
2. **PDF validation.** The system checks that the file is a PDF. A valid PDF is accepted. Any other file is rejected and is not processed further.
3. **SHA-256 hash generation.** The hash of the accepted file is calculated. The result is a 64-character hexadecimal value.
4. **Storage.** The document details and the hash are stored in MongoDB and linked to the property.

5. **Listing.** The document appears in the document list with its property, file name, file size, shortened hash and upload time.
6. **Verification when required.** When a copy of the document is submitted later, its hash is calculated and compared with the registered hash.

In the proposed architecture, one more action follows step 3: the hash is also recorded on the Hyperledger Fabric ledger together with the property ID, document ID and timestamp.

4.2.1 Algorithms and Methodology

SHA-256 hashing: SHA-256 is a member of the SHA-2 family of standard hash functions. It takes input of any length and produces a 256-bit output, represented as 64 hexadecimal characters. The same file produces the same hash when processed again, while even a small change in the file produces a different hash. This property makes SHA-256 suitable for checking whether a submitted document matches the registered document. The hash is also one-way, meaning that the original document cannot practically be reconstructed from the hash.

Document hash generation: When a document is uploaded, the system receives the selected property and file, checks whether the file is a PDF, calculates its SHA-256 hash, stores the document details and hash in MongoDB, and returns the 64-character hash to the user interface for display.

Document verification: During verification, the submitted document is received and its SHA-256 hash is calculated. The system then looks up the registered document record. If no registered record exists, the result is **NOT FOUND**. If the computed hash is equal to the registered hash, the result is **MATCH**. Otherwise, the result is **MISMATCH**, and the computed and registered hashes are displayed for comparison.

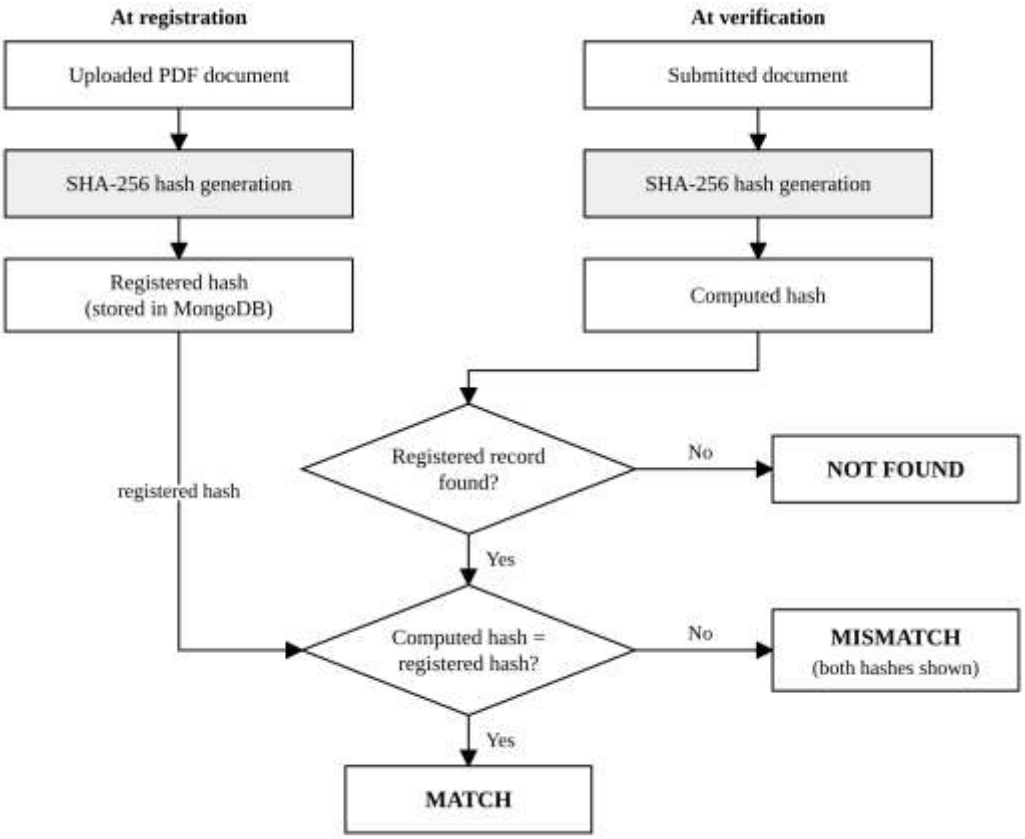


Figure 4.2 Hash comparison during verification

Property record handling: A property is saved only when its Property ID is not already present in the property records. The system checks the Property ID before saving. If it already exists, the entry is rejected. Otherwise, the property record is saved, the form is cleared and the new property appears in the property list. Pasted markdown

Conflict detection methodology: Conflict detection in this project is rule-based. Each rule is defined as a condition that can be checked against stored records, making the reason for a conflict clear. No machine learning model is used. Duplicate Property ID detection is implemented and tested, while duplicate survey number, owner mismatch, area mismatch, invalid ownership transfer and duplicate document hash are proposed rules. Pasted markdown

The exact conditions of the proposed rules will be fixed when they are implemented. In the proposed design, a record that triggers a rule is marked as a potential conflict and sent to an authorised officer for review. The system

does not decide ownership and does not label a record as fraudulent. Pasted markdown

Authentication: The administrator logs in through the login page, and the submitted details are checked by the application. A wrong password is rejected, while a correct login leads to the dashboard.

4.2.2 RESULTS AND DISCUSSION

The application was tested for correct functioning. Backend API testing was completed successfully. The user interface was tested in a real Google Chrome browser, and all 12 UI test cases passed. The frontend build and backend start-up also completed successfully.

The authentication test showed that an invalid password was rejected, while valid login details opened the dashboard. The dashboard displayed five statistic cards and three recent-record tables. Property registration was successful for a new Property ID, and duplicate Property IDs were rejected. Valid PDF files were accepted, while non-PDF files were rejected.

The document hashing test confirmed that a complete 64-character SHA-256 hash was generated and displayed. The document list displayed the property, file name, file size, shortened hash and upload time.

The three verification cases produced the expected results. The original registered document produced **MATCH**, confirming that the calculated hash was equal to the registered hash. A modified copy produced **MISMATCH**, with the computed and registered hashes displayed side by side. An unregistered document produced **NOT FOUND**, indicating that no registered record was available for comparison.

The main screens of the application are shown in Figures 4.3 to 4.6. Figure 4.3 shows the admin login page and Figure 4.4 shows the dashboard with its five statistic cards and three tables of recent records. Figure 4.5 shows the property registration page, and Figure 4.6 shows a document upload with the

generated 64-character hash. The document verification results are discussed separately in the results section.

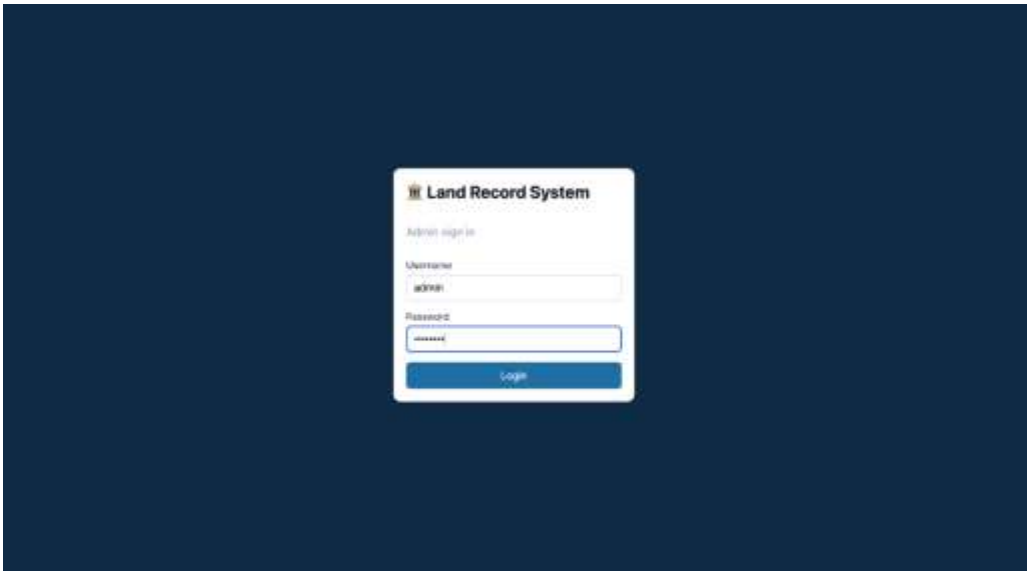


Figure 4.3 Admin login page

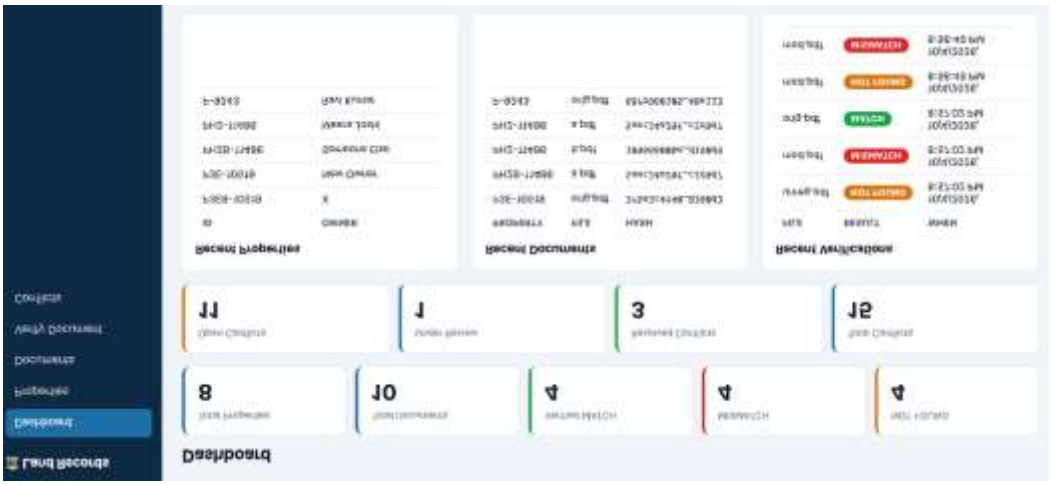


Figure 4.4 Dashboard with statistic cards and recent records

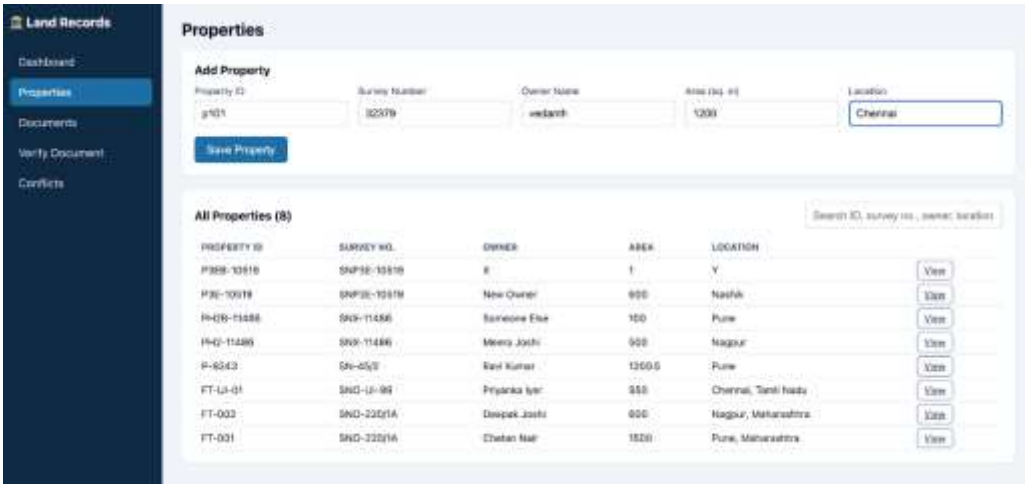


Figure 4.5 Property registration

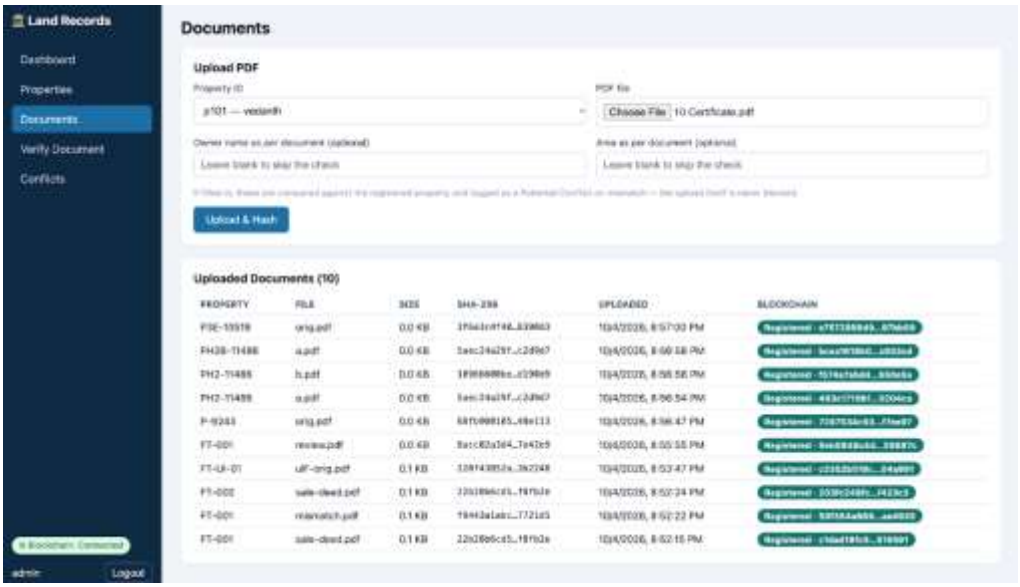


Figure 4.6 Document upload and generated hash

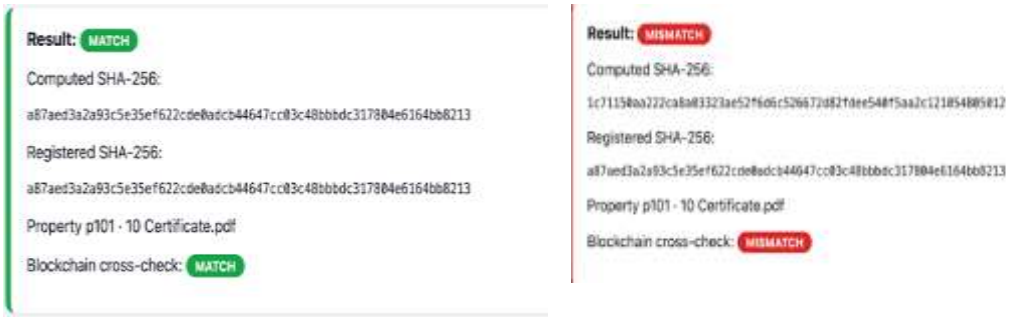


Figure 4.7 Verification result MATCH & MISMATCH

Discussion: The The results show that the core function of the project works as intended. The original document gave **MATCH**, which confirms that the hash is calculated consistently during upload and verification. The modified

document gave **MISMATCH**, which shows that a change in the content of the file is detected. Showing the computed and registered hashes side by side makes the reason for the result visible to the user. The unregistered document gave **NOT FOUND**, so a document that was never registered is not confused with a registered document that has been changed.

The input checks also worked. An invalid password did not give access, a non-PDF file was not accepted, and a duplicate Property ID was not saved. These checks stop wrong or unwanted data at the point of entry. The persistence test showed that the records and hashes are kept in MongoDB and are not lost when the database and backend are restarted.

The results have clear limits. The tests check correct functioning only. No performance values such as response time or throughput were measured, and no load or penetration testing was done. The registered hash is stored in MongoDB, so the verification depends on the database not being altered. The results do not include Hyperledger Fabric transactions. Of the conflict rules, only the duplicate Property ID rule is active.

Compared with the reviewed work, the verification function follows the hash comparison principle described in DOC-BLOCK (Imam et al., 2021). In the present project, the document is linked to a property record, and the result separates a changed document from an unregistered one. The land registration frameworks of Khanna et al. (2026) and Sen et al. (2021) report performance measurements for their blockchain networks. Similar measurements can be carried out for this project after the proposed blockchain component is integrated.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 CONCLUSION

This project aimed to make land documents easier to verify and conflicting records easier to identify. A web application was developed using React and Vite, Node.js and Express, and MongoDB. The system registers properties, accepts land-related PDF documents, generates a SHA-256 hash for each document and stores the hash with the document details. A submitted document can later be verified, and the result is reported as **MATCH, MISMATCH or NOT FOUND**.

Testing confirmed the expected behaviour. Invalid login attempts were rejected, duplicate Property IDs and non-PDF files were refused, SHA-256 hashes were generated and displayed as 64 hexadecimal characters, and all three verification results were obtained correctly. The proposed architecture further extends the system with Hyperledger Fabric, rule-based conflict checks and officer review. The current system does not replace the existing registration process or decide legal ownership. A **MATCH** result indicates only that the submitted file is identical to the registered file.

5.2 FUTURE ENHANCEMENTS

Three enhancements are identified for future work. The first is to integrate the Hyperledger Fabric network and chaincode with the backend, so that hashes and transaction details are recorded on the ledger. The second is to implement the other conflict rules, the officer review workflow, the ownership transaction history and separate roles for verifiers and officers. The third is to measure performance after the blockchain component is integrated.

REFERENCES

1. E. M. Alotaibi, H. Issa, and M. Codesso, "Blockchain-based conceptual model for enhanced transparency in government records: A design science research approach," *International Journal of Information Management Data Insights*, vol. 5, no. 1, Art. no. 100304, 2025, doi: 10.1016/j.jjimei.2024.100304.

2. L. Dcunha, L. Dsouza, G. Ramanan, and S. Chaudhari, "A blockchain-based approach to streamlining land registration using Hyperledger Fabric," in *Proc. 2023 International Conference on Emerging Smart Computing and Informatics (ESCI)*, 2023, doi: 10.1109/ESCI56872.2023.10099592.

3. I. T. Imam, Y. Arafat, K. S. Alam, and S. A. Shahriyar, "DOC-BLOCK: A blockchain based authentication system for digital documents," in *Proc. 2021 3rd International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)*, 2021, pp. 1262–1267, doi: 10.1109/ICICV50876.2021.9388428.

4. C.-I. Irimia and A. Iftene, "Decentralized infrastructure for digital notarizing, signing and sharing documents securely using microservices and blockchain," *IEEE Access*, vol. 12, 2024, doi: 10.1109/ACCESS.2024.3518977.

5. A. Khanna, P. Singh, and P. K. Angra, "Blockchain-enabled e-governance framework for secure and transparent land record management," *International Research Journal of Multidisciplinary Technovation*, vol. 8, no. 4, pp. 73–90, 2026, doi: 10.54392/irjmt2645.

6. K. Kumutha and S. Jayalakshmi, "Hyperledger Fabric blockchain framework: Efficient solution for academic certificate decentralized repository," in *Proc. 2021 5th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)*, 2021, pp. 1584–1590, doi: 10.1109/I-SMAC52330.2021.9640785.

7. Y. Liu and Y. Chang, "Blockchain-based method for spatial retrieval and verification of remote sensing images," *Sensors*, vol. 24, no. 7, Art. no. 2078, 2024, doi: 10.3390/s24072078.

8. P. Mishra and R. Ganesan, "Securing e-governance against shadow attacks with blockchain technology," *Scientific Reports*, vol. 15, Art. no. 42306, 2025, doi: 10.1038/s41598-025-26326-0.

9. S. R. Nagaraja and R. Shushma, "A hybrid AI-blockchain framework with OCR, NER, and anomaly detection for secure land record management," *Engineering, Technology & Applied Science Research*, vol. 15, no. 6, pp. 30252–30259, 2025, doi: 10.48084/etasr.15223.

10. F. Naguji, N. K. Jadav, S. Tanwar, G. Pau, G. Sharma, F. Alqahtani, and A. Tolba, “GreenLand: A secure land registration scheme for blockchain and AI-enabled agriculture Industry 5.0,” *IEEE Access*, vol. 12, pp. 120994–121009, 2024, doi: 10.1109/ACCESS.2024.3451627.

11. O. U. Nwankwo, M. R. R. Ansori, E. A. Tuli, D.-S. Kim, and J.-M. Lee, “BERBLOM: Blockchain-enabled reliable land ownership management system using ERC-4907 smart contract,” *Peer-to-Peer Networking and Applications*, 2026, doi: 10.1007/s12083-026-02227-5.

12. S. N. Pari, S. Rajashree, A. Prakash, and R. M. Shanthosh, “Role based access control framework for healthcare records using Hyperledger Fabric,” in *Proc. 2022 International Conference on Innovative Computing, Intelligent Communication (ICICT)*, 2022.

13. S. Sen, S. Mukhopadhyay, and S. Karforma, “A blockchain based framework for property registration system in e-governance,” *International Journal of Information Engineering and Electronic Business*, vol. 13, no. 4, pp. 30–46, 2021, doi: 10.5815/ijieeb.2021.04.03.

14. M. Stefanović, Đ. Pržulj, S. Ristić, D. Stefanović, and D. Nikolić, “Smart contract application for managing land administration system transactions,” *IEEE Access*, vol. 10, pp. 39154–39176, 2022, doi: 10.1109/ACCESS.2022.3164444.

15. R. M. Zein and H. Twinomurinzi, “Information sharing in land registration using Hyperledger Fabric blockchain,” *Blockchains*, vol. 2, no. 2, pp. 107–133, 2024, doi: 10.3390/blockchains2020006.

APPENDIX 1

PHASE I – TEST CASES AND RESULTS

Phase I of the proposed system was tested to validate the core functionalities of administrator authentication, property registration, document upload, SHA-256 hash generation, document verification, and database persistence. The administrator login was tested using both valid and invalid credentials, with successful authentication for valid credentials and rejection of invalid login attempts. Property registration was verified by adding a new property and checking its availability in the system, while duplicate Property IDs were correctly rejected. The document management functionality was tested by uploading valid PDF files and attempting to upload unsupported file types. For each valid document, the system generated a 64-character SHA-256 hash that served as the digital fingerprint of the document.

The document verification functionality was evaluated using original, modified, and unregistered documents. The original registered document produced a MATCH result because its generated hash was identical to the registered hash. A modified version of the document produced a MISMATCH result because the document content generated a different SHA-256 hash. An unregistered document produced a NOT FOUND result because no corresponding registered document existed in the system. Database persistence was also verified by restarting the application and confirming that the previously registered property and document information remained available. The Phase I testing results demonstrate that the implemented system successfully performs property registration, document management, cryptographic hash generation, document integrity verification, and persistent data storage, thereby establishing the functional foundation for the subsequent phases of the project.